Pedro Henrique Gomes

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

São Paulo
2026

Pedro Henrique Gomes

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Prof. Afonso Brandão

São Paulo
2026

Resumo

A crescente disseminação de dispositivos de exibição com elevadas taxas de atualização (60 Hz a 144 Hz) evidenciou a limitação temporal de conteúdos audiovisuais capturados nas frequências tradicionais de 24 FPS a 30 FPS. Para mitigar artefatos visuais e aumentar a fluidez contínua, a Interpolação de Quadros de Vídeo (VFI – *Video Frame Interpolation*) busca sintetizar quadros intermediários. Embora abordagens baseadas em fluxo óptico denso, como o modelo RAFT, alcancem elevada precisão, a alta complexidade computacional e a dependência de *frameworks* de alto nível (PyTorch) limitam sua aplicação sob restrições severas de hardware. Como alternativa eficiente, as Convoluções Adaptativas Separáveis tratam a síntese como uma filtragem local adaptativa, decompondo *kernels* bidimensionais $N \times N$ em dois filtros unidimensionais $1D$, reduzindo a complexidade aritmética de $O(N^2)$ para $O(2N)$ por pixel. Este trabalho apresenta uma análise experimental do modelo SepConv através da implementação de um motor de inferência nativo em C++ e CUDA, desacoplado de ecossistemas de treinamento. O *pipeline* proposto integra a biblioteca OpenCV para entrada e saída de dados, a API C++ do ONNX Runtime para a extração dos vetores de movimento e *kernels* customizados em CUDA para a execução paralela das convoluções mantidas diretamente na memória de vídeo (VRAM). A validação experimental prevê a avaliação do compromisso entre o consumo de memória VRAM (aferido via NVML e focado em GPUs de consumo de 8 GB), a latência de execução em milissegundos e a fidelidade visual (mensurada por PSNR e SSIM sobre o *dataset* Vimeo-90K). Os resultados buscam comprovar a viabilidade técnica do controle granular de memória para viabilizar a interpolação de vídeos em alta definição em hardwares comerciais restritos.

Palavras-chave: Interpolação de Quadros de Vídeo. Convoluções Adaptativas Separáveis. C++. CUDA. Otimização de VRAM. SepConv.

Sumário

Sumário 3

1 INTRODUÇÃO 4
1.1 Contexto 5
1.2 Justificativa 6
1.3 Objetivo 6
1.3.1 Objetivo principal 7
1.3.2 Objetivos secundários 7

2 REVISÃO BIBLIOGRÁFICA 8
2.1 Metodologia da pesquisa bibliográfica 8
2.1.1 Período de abrangência 8
2.1.2 Critérios de inclusão 9
2.1.3 Critérios de exclusão 9
2.2 Fundamentos 10
2.2.1 Definição de Imagem Digital 10
2.2.2 Filtragem Espacial e Convolução Adaptativa 11
2.2.3 Fluxo Óptico Explícito vs. Kernels de Movimento Implícito 12
2.2.4 Eficiência via Convoluções Separáveis 13
2.2.5 Implementação de Baixo Nível e I/O de Vídeo 14
2.2.6 Inferência Desacoplada via ONNX Runtime 15
2.2.7 Arquitetura de Decomposição e Garantia de Separabilidade 15
2.2.8 Arquitetura de GPU e o Gargalo de Memória (*Memory Wall*) 16
2.2.9 Métricas de Avaliação Objetiva de Qualidade 17
2.2.10 Mensuração de Desempenho Computacional 18
2.3 Trabalhos relacionados 19
2.3.1 Trabalho 1: Revisiting Adaptive Convolutions for Video Frame Interpolation 19
2.3.2 Trabalho 2: Motion-Aware Video Frame Interpolation 19
2.3.3 Trabalho 3: RAFT: Recurrent All-Pairs Field Transforms for Optical Flow . 20

3 DESENVOLVIMENTO 22
3.1 Solução proposta 22
3.2 Cronograma de trabalho 23

REFERÊNCIAS 25

1 Introdução

Historicamente, os sistemas de captura e exibição de imagens em movimento foram limitados pela capacidade mecânica e elétrica dos dispositivos da época. Durante a consolidação do cinema sonoro no início do século XX, a taxa de aproximadamente 24 quadros por segundo (FPS) tornou-se um padrão industrial por representar um equilíbrio entre fluidez perceptiva e custo de armazenamento e reprodução do filme físico (BORDWELL; THOMPSON; SMITH, 2016). Taxas nessa faixa passaram a ser suficientes para produzir a percepção de movimento contínuo ao sistema visual humano, reduzindo a percepção de descontinuidade temporal durante a projeção (WATKINSON, 2009).

Com a evolução tecnológica dos dispositivos de exibição, monitores e televisores modernos passaram a operar em frequências significativamente superiores, variando entre 60 Hz e 144 Hz. Entretanto, grande parte do conteúdo audiovisual disponível ainda permanece limitada às taxas tradicionais de captura, como 24 FPS ou 30 FPS, criando uma discrepância entre a capacidade de atualização do hardware atual e a quantidade de informação temporal presente no vídeo original. Essa diferença entre a taxa de amostragem temporal do conteúdo original e a frequência de atualização do dispositivo de exibição resulta no fenômeno conhecido como subamostragem temporal. De acordo com Watkinson (2009) e Szeliski (2022), a incapacidade de fornecer amostras intermediárias suficientes durante movimentos rápidos reduz a percepção de continuidade temporal pelo sistema visual humano, induzindo a artefatos perceptuais como tremulação de movimento (*motion judder*), borramento dinâmico e quebra na percepção de fluidez contínua.

Nesse contexto, a Interpolação de Quadros de Vídeo (VFI – Video Frame Interpolation) surge como uma técnica essencial para sintetizar quadros intermediários artificialmente, aumentando a fluidez temporal do conteúdo exibido. Tradicionalmente, a literatura divide as abordagens de VFI em métodos baseados em fluxo óptico, como o modelo RAFT (TEED; DENG, 2020), e métodos baseados em kernels. Embora modelos de aprendizado profundo tenham atingido patamares elevados de precisão, eles frequentemente demandam recursos computacionais extensivos e dependem de frameworks pesados, o que limita sua aplicação em cenários de hardware restrito ou escassez de memória RAM e VRAM. Para mitigar a alta demanda de recursos computacionais, representada pelo elevado volume de operações de ponto flutuante (FLOPs), pelo esgotamento do armazenamento de tensores na memória de vídeo (VRAM) e pela alta latência de execução —, a literatura recente busca alternativas mais eficientes aos modelos tradicionais de fluxo óptico. Em cenários de hardware restrito, típicos de computadores pessoais e dispositivos de consumo doméstico dotados de GPUs comerciais limitadas a 8 GB de VRAM, a execução de ecossistemas densos baseados em *frameworks* de alto nível (como PyTorch) torna-se inviável em altas resoluções. Nesse contexto, Niklaus et al. (2017) propuseram a utilização de Convoluções

Adaptativas Separáveis, técnica que decompoe *kernels* bidimensionais $N \times N$ em pares de filtros unidimensionais $1D$, reduzindo a complexidade computacional de $O(N^2)$ para $O(2N)$. A implementação dessa etapa de inferência em linguagem de baixo nível (C++/CUDA) possibilita um controle granular sobre o gerenciamento de memória gráfica, permitindo isolar a eficiência do algoritmo e testar sua viabilidade em computadores comuns. Diante desse gargalo tecnológico entre a necessidade de fluidez em computadores pessoais e as limitações físicas de memória do hardware de consumo, formaliza-se a seguinte **hipótese científica**: postula-se que a separabilidade matemática dos *kernels*, quando associada a um *pipeline* de inferência nativo em C++ e CUDA (desacoplado de *frameworks* de treinamento), reduz o pico de consumo de VRAM e a latência de execução em comparação às implementações de alto nível em Python/PyTorch. A partir dessa premissa, o presente trabalho tem como objetivo avaliar a viabilidade técnica do processamento de vídeos em alta definição sob o limite de 8 GB de VRAM, investigando o equilíbrio entre a eficiência computacional no hardware do usuário final e a fidelidade visual dos quadros sintetizados.

1.1 Contexto

Historicamente, os padrões de captura e transmissão de vídeo foram consolidados sob limitações técnicas de largura de banda e processamento da época, estabelecendo taxas como 24 ou 30 quadros por segundo (FPS) como normas globais (GONZALEZ; WOODS, 2009). Contudo, a evolução recente dos dispositivos de exibição permitiu o surgimento de telas com altas taxas de atualização (60Hz a 144Hz), criando uma disparidade entre o conteúdo disponível e a capacidade do hardware. Essa lacuna resulta em uma percepção de movimento descontínua, pois a amostragem temporal reduzida do conteúdo original não preenche os intervalos de atualização das telas modernas, comprometendo a fluidez visual esperada pelo usuário.

Diferente de métodos simplificados de amostragem de quadros, a VFI busca sintetizar um novo quadro no instante $t+0.5$ por meio de uma combinação ponderada de informações espaciais e temporais das imagens adjacentes. Esse processo considera o contexto do movimento e a correlação entre pixels (NIKLAUS; MAI; LIU, 2017). Ao elevar a taxa de quadros por segundo, a técnica objetiva atenuar artefatos de movimento na percepção humana.

Entretanto, a implantação prática desses algoritmos enfrenta barreiras atreladas ao custo computacional espacial e aritmético. Modelos de estado da arte demandam alto armazenamento de tensores intermediários na memória de vídeo (VRAM), limitando frequentemente sua execução a hardwares de alto desempenho (BAO et al., 2019). A transposição de modelos matemáticos para pipelines de inferência otimizados surge como um caminho científico relevante para avaliar os reais gargalos de desempenho em sistemas com memória de consumo limitada.

1.2 Justificativa

A escolha pela implementação de algoritmos de interpolação baseados em Convoluções Adaptativas Separáveis justifica-se, primordialmente, pela necessidade de viabilizar a fluidez visual em sistemas de computação pessoal com recursos de memória limitados. Enquanto as abordagens de convolução espacial completa exigem uma complexidade quadrática de N^2 operações por pixel para a síntese de quadros, a técnica de decomposição em *kernels* unidimensionais reduz esse custo para uma ordem linear de $2N$ (NIKLAUS; MAI; LIU, 2017). Essa eficiência matemática busca diminuir o tempo de execução e, essencialmente, o *footprint* (consumo espacial) na memória de vídeo (VRAM), viabilizando que hardwares de consumo processem sequências de vídeo em alta definição sem estourar os limites da placa gráfica.

A delimitação experimental em GPUs de consumo dotadas de 8 GB de memória VRAM fundamenta-se na ampla representatividade desse perfil de hardware no mercado do usuário final. Segundo os levantamentos da *Steam Hardware & Software Survey* (Valve Corporation, 2026), placas de vídeo com 8 GB de VRAM representam aproximadamente 26% das configurações ativas na plataforma, constituindo a capacidade individual mais expressiva entre os usuários domésticos. Embora modelos de alto desempenho com 16 GB ou mais tenham registrado crescimento recente, a faixa de 8 GB permanece como a principal barreira física para a execução de modelos densos de inteligência artificial em ambientes não corporativos.

Sob a perspectiva da Ciência da Computação e da Engenharia de Software, a pesquisa justifica-se pela investigação do gargalo de abstração imposto por *frameworks* de alto nível (como Python/PyTorch). Embora adequados para prototipagem e treinamento, esses ecossistemas introduzem *overheads* significativos decorrentes do gerenciamento dinâmico de memória, da alocação redundante de tensores temporários e da camada de interpretação do runtime. Ao transpor a etapa de inferência do modelo SepConv diretamente para a linguagem C++, integrada ao ecossistema nativo ONNX Runtime e a *kernels* customizados em CUDA, é possível exercer um controle determinístico sobre a alocação e transferência de ponteiros na VRAM.

Dessa forma, o trabalho busca demonstrar como o gerenciamento granular de hardware e a otimização algorítmica de baixo nível conseguem mitigar limitações físicas de memória gráfica, oferecendo um ponto de equilíbrio viável entre a qualidade perceptual da imagem e o desempenho computacional em sistemas comerciais.

1.3 Objetivo

Esta seção delimita o escopo técnico da pesquisa, focando na avaliação de desempenho sob restrições físicas de VRAM em hardware de consumo.

1.3.1 Objetivo principal

Avaliar experimentalmente os efeitos da convolução separável no desempenho computacional da inferência do modelo SepConv em GPU, utilizando uma implementação nativa em C++, com ênfase na latência e no consumo de memória VRAM em hardware de consumo.

1.3.2 Objetivos secundários

1. Desenvolver um fluxo de processamento em C++ e OpenCV para a extração de quadros e transferência eficiente para a memória da placa de vídeo;

2. Integrar a biblioteca *ONNX Runtime* em C++ para carregar os pesos pré-treinados do modelo SepConv e inferir os vetores separáveis (1D) correspondentes ao movimento;

3. Codificar e otimizar *kernels* customizados em arquitetura CUDA para aplicar a operação de convolução separável diretamente na VRAM;

4. Comparar o desempenho da implementação proposta (C++) exclusivamente com a *baseline* da implementação original do SepConv (Python/PyTorch), com ênfase na aferição da redução do pico de consumo de memória VRAM e na latência de execução;

5. Mensurar o custo computacional e a viabilidade de execução com baixa latência, analisando o tempo de execução, a taxa de quadros por segundo (FPS) e o pico de uso de memória VRAM em hardware de consumo para diferentes resoluções de vídeo.

2 Revisão Bibliográfica

A revisão bibliográfica tem como finalidade apresentar o embasamento teórico do trabalho, demonstrando o conhecimento do autor sobre o tema e identificando o estado da arte. Este capítulo deve ser organizado por conceitos e temas, construindo uma linha de argumentação que sustente a proposta do trabalho.

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida por meio de buscas nas bases Google Scholar e IEEE Xplore, utilizando combinações de palavras-chave relacionadas ao tema da pesquisa. As consultas empregaram operadores lógicos para refinar os resultados, buscando trabalhos relacionados à interpolação de quadros de vídeo, convoluções separáveis, fluxo óptico e otimização de inferência em GPU.

- **Bases de dados consultadas**: Google Scholar e IEEE Xplore;
- **Palavras-chave utilizadas**:
  - ("video frame interpolation"OR "frame interpolation") AND ("convolutional neural network"OR CNN OR convolution) AND (video OR image sequence);
  - ("video frame interpolation") AND (convolution OR "adaptive convolution"OR "separable convolution");
  - ("video frame interpolation") AND ("optical flow"OR convolution OR transformer OR "deep learning");

2.1.1 Período de abrangência

Embora as buscas tenham retornado publicações de diferentes períodos, foi adotado como critério principal a seleção de trabalhos publicados a partir de 2015. Esse recorte temporal foi definido por contemplar o surgimento e a consolidação das principais técnicas modernas de aprendizado profundo aplicadas à interpolação de quadros de vídeo. Trabalhos anteriores foram mantidos apenas quando considerados referências clássicas ou fundamentais para a compreensão dos conceitos teóricos utilizados nesta pesquisa.

O processo de busca inicial retornou aproximadamente 5.000 resultados. Em uma primeira etapa de triagem (*screening*), foi realizada a leitura dos títulos dos trabalhos encontrados, resultando na seleção de aproximadamente 60 publicações potencialmente relevantes ao tema investigado. Após a aplicação do filtro temporal, cerca de 20 artigos permaneceram para análise.

Na etapa seguinte, foi realizada a leitura dos resumos dos trabalhos selecionados, reduzindo o conjunto para aproximadamente 12 publicações. Por fim, foi conduzida uma leitura mais aprofundada dos artigos com maior aderência ao escopo da pesquisa, dos quais cerca de cinco foram efetivamente utilizados na construção do referencial teórico e três foram selecionados para compor a seção de trabalhos relacionados.

2.1.2 Critérios de inclusão

Foram considerados elegíveis os trabalhos que atendessem a pelo menos um dos seguintes critérios:

- abordagem relacionada à interpolação de quadros de vídeo (*Video Frame Interpolation*);
- utilização de convoluções adaptativas ou convoluções separáveis;
- aplicação de técnicas de fluxo óptico para síntese temporal de quadros;
- estudos envolvendo inferência acelerada por GPU;
- pesquisas relacionadas à otimização computacional, consumo de memória ou redução de latência em aplicações de visão computacional;
- publicações com relevância científica comprovada por meio de conferências ou periódicos reconhecidos na área.

2.1.3 Critérios de exclusão

Foram excluídos da análise trabalhos que apresentassem uma ou mais das seguintes características:

- baixa aderência ao problema investigado;
- foco exclusivo na etapa de treinamento de modelos neurais;
- dependência excessiva de hardware específico sem aplicabilidade ao contexto da pesquisa;
- ausência de descrição técnica suficiente para reprodução ou compreensão da metodologia proposta;
- duplicidade entre bases consultadas;
- trabalhos que não abordassem aspectos relacionados à interpolação de quadros, fluxo óptico, convoluções separáveis ou otimização de inferência.

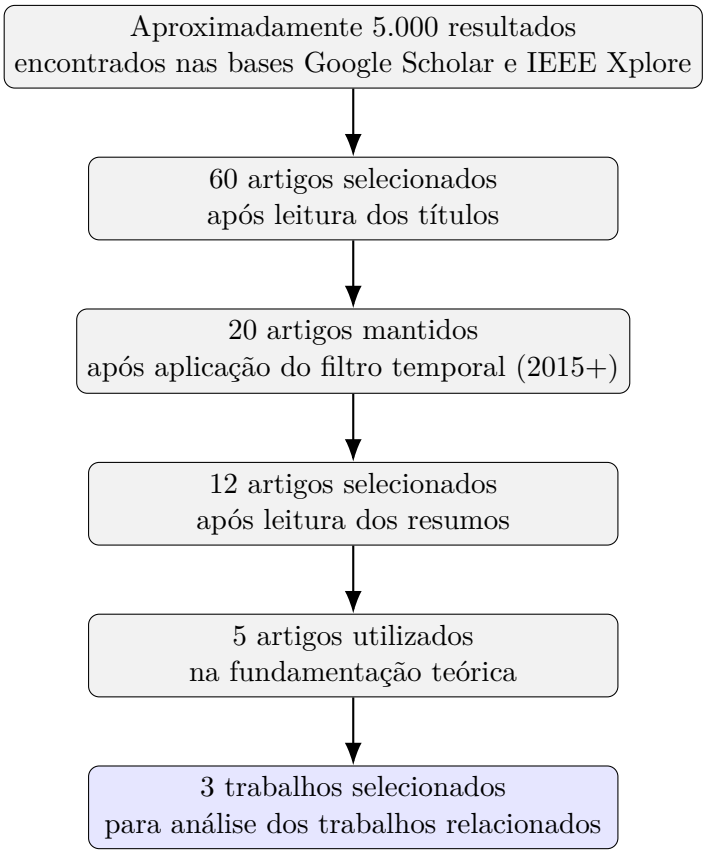


Figura 1 – Processo de seleção dos trabalhos utilizados na pesquisa bibliográfica.

Fonte: Elaborado pelo autor (2026)

A estratégia de seleção adotada permitiu reduzir progressivamente o conjunto inicial de publicações, mantendo apenas trabalhos com elevada aderência técnica ao objetivo da pesquisa e representativos do estado da arte em interpolação de quadros de vídeo e otimização de inferência em GPU.

2.2 Fundamentos

2.2.1 Definição de Imagem Digital

Uma imagem digital pode ser definida como uma função bidimensional $f(x, y)$, onde x e y representam coordenadas espaciais em um plano. De acordo com Gonzalez e Woods (2009), quando os valores de x, y e a amplitude de f assumem quantidades finitas e discretas, a imagem é caracterizada como digital. Na prática computacional, essa estrutura é representada por uma matriz composta por elementos discretos denominados *pixels* (*picture elements*), que constituem a menor unidade de informação visual de uma imagem.

No modelo de cores RGB (*Red, Green, Blue*), amplamente utilizado em sistemas computacionais, cada pixel é representado por três componentes correspondentes aos

canais vermelho, verde e azul. Em implementações convencionais de 8 bits por canal, cada componente assume valores no intervalo $[0, 255]$, permitindo a representação de aproximadamente 16,7 milhões de combinações de cores (SZELISKI, 2022).

A resolução de uma imagem corresponde à quantidade total de pixels que a compõem. Em aplicações de vídeo, o volume de dados cresce rapidamente com o aumento da resolução. Por exemplo, um único quadro em resolução Full HD (1920×1080) contém mais de dois milhões de pixels. Considerando três canais de cor por pixel, a manipulação simultânea de múltiplos quadros pode demandar dezenas de megabytes de memória, tornando o gerenciamento eficiente dos dados um aspecto relevante para algoritmos de processamento de vídeo e interpolação de quadros (SZELISKI, 2022).

A representação dessas matrizes em linguagem C++ impacta diretamente a performance computacional. Embora conceitualmente bidimensionais, as imagens são frequentemente organizadas na memória como vetores unidimensionais para maximizar o acesso contíguo aos dados. Nesse formato de linearização, a posição de um pixel (x, y) em uma imagem de largura W pode ser obtida pela expressão:

$$index = (y \times W) + x$$

Essa organização favorece a localidade espacial de memória e reduz a ocorrência de acessos não contíguos, característica particularmente importante em aplicações de processamento paralelo executadas em GPUs (SZELISKI, 2022; KIRK; HWU, 2023). Em algoritmos de interpolação de quadros, nos quais milhões de operações são realizadas para cada sequência de vídeo processada, a forma de armazenamento dos pixels pode influenciar diretamente a latência de execução e o consumo de memória.

2.2.2 Filtragem Espacial e Convolução Adaptativa

A convolução constitui uma das operações fundamentais do processamento digital de imagens e da visão computacional moderna. Em sua forma tradicional, um kernel bidimensional percorre toda a imagem aplicando o mesmo conjunto de coeficientes a cada região analisada. Segundo Gonzalez e Woods (2009), filtros convolucionais clássicos utilizam coeficientes fixos para executar tarefas como suavização, realce de detalhes, remoção de ruído e detecção de bordas.

Matematicamente, a convolução espacial pode ser interpretada como uma operação de combinação ponderada entre os pixels de uma vizinhança local. Em aplicações convencionais de processamento de imagens, a utilização de um kernel fixo é suficiente, uma vez que a transformação desejada é aplicada uniformemente sobre toda a cena. Entretanto, essa abordagem torna-se limitada em problemas de Interpolação de Quadros de Vídeo (VFI), nos quais diferentes regiões da imagem podem apresentar movimentos distintos entre quadros consecutivos (SZELISKI, 2022).

Em cenários de VFI, o objetivo não consiste apenas em filtrar uma imagem existente, mas em sintetizar novas informações visuais correspondentes a um instante temporal intermediário. Dessa forma, um único kernel fixo não é capaz de representar adequadamente a variedade de movimentos presentes na cena. Objetos distintos podem deslocar-se em velocidades e direções diferentes, exigindo estratégias adaptativas para estimar corretamente a posição dos pixels no quadro intermediário.

Para superar essa limitação, surgem as convoluções adaptativas, nas quais os coeficientes do kernel deixam de ser constantes e passam a ser estimados individualmente para cada pixel da imagem. Conforme proposto por Niklaus, Mai e Liu (2017), os coeficientes desses kernels funcionam como indicadores de amostragem, determinando de quais posições espaciais nos quadros de entrada as informações devem ser extraídas para sintetizar o pixel correspondente ao instante intermediário $(t + 0.5)$.

Essa estratégia permite incorporar implicitamente informações de movimento durante o próprio processo de síntese da imagem. Em vez de realizar uma etapa explícita de estimativa de fluxo óptico seguida de reconstrução do quadro, a convolução adaptativa unifica ambas as tarefas em uma única operação. Como resultado, o modelo passa a aprender simultaneamente onde buscar informações e como combiná-las, reduzindo a complexidade do pipeline e possibilitando a geração de quadros intermediários com elevada qualidade visual (NIKLAUS, 2021).

2.2.3 Fluxo Óptico Explícito vs. Kernels de Movimento Implícito

Grande parte dos métodos modernos de Interpolação de Quadros de Vídeo (VFI) baseia-se na estimativa de movimento entre quadros consecutivos. Nesse contexto, o conceito de fluxo óptico desempenha papel fundamental ao descrever o deslocamento aparente dos pixels entre duas imagens ao longo do tempo. Formalmente, o fluxo óptico pode ser representado por um campo vetorial denso, no qual cada pixel possui um vetor (u, v) indicando sua direção e magnitude de deslocamento (SZELISKI, 2022).

Entre os modelos contemporâneos mais relevantes para estimativa de fluxo óptico destaca-se o RAFT (*Recurrent All-Pairs Field Transforms*), proposto por Teed e Deng (2020). O modelo constrói volumes de correlação entre todos os pares de pixels das imagens de entrada e utiliza um mecanismo recorrente para refinar iterativamente os vetores de movimento. Essa estratégia permite atingir elevados níveis de precisão, tornando o RAFT uma das principais referências da área de visão computacional.

Após a estimativa do fluxo óptico, métodos tradicionais de VFI realizam uma etapa adicional denominada *warping*, responsável por deformar ou reposicionar os pixels dos quadros originais de acordo com os vetores calculados. Embora eficaz, essa abordagem pode apresentar dificuldades em regiões de oclusão, desoclusão ou movimentos complexos, nas quais determinados pixels tornam-se visíveis ou invisíveis entre quadros consecutivos (TEED; DENG, 2020; SZELISKI, 2022).

Uma alternativa a esse paradigma consiste na utilização de kernels de movimento implícito, abordagem adotada pelos métodos baseados em convolução adaptativa. Nesse cenário, o movimento não é representado explicitamente por vetores de deslocamento. Em vez disso, a rede neural aprende diretamente um conjunto de coeficientes responsáveis por determinar como os pixels dos quadros de entrada devem ser combinados para sintetizar o valor correspondente ao quadro intermediário (NIKLAUS; MAI; LIU, 2017).

Sob essa perspectiva, os pesos do kernel passam a codificar implicitamente informações de movimento e correspondência espacial. Em vez de calcular uma "seta"indicando o deslocamento de cada pixel, o modelo aprende quais amostras locais devem ser utilizadas e qual contribuição cada uma delas terá na reconstrução do novo quadro. Dessa forma, a estimativa de movimento e a síntese da imagem tornam-se partes de uma única operação computacional.

Essa unificação simplifica o *pipeline* de processamento e elimina a necessidade de etapas explícitas de fluxo óptico e *warping*. Além disso, reduz a propagação de erros entre estágios sucessivos do algoritmo, característica que contribuiu para a popularização das abordagens baseadas em convolução adaptativa na literatura recente de interpolação de quadros de vídeo (NIKLAUS; MAI; LIU, 2017; NIKLAUS, 2021).

2.2.4 Eficiência via Convoluções Separáveis

O principal desafio das abordagens baseadas em convolução adaptativa está relacionado ao tamanho dos kernels necessários para representar movimentos de grande magnitude. Em métodos de interpolação de quadros, cada pixel do quadro intermediário pode depender de uma ampla região dos quadros de entrada, exigindo filtros com dimensões elevadas. Consequentemente, o custo computacional cresce rapidamente à medida que o tamanho do kernel aumenta (NIKLAUS; MAI; LIU, 2017).

Em uma convolução bidimensional convencional, um kernel de dimensão $N \times N$ contém N^2 coeficientes. Isso significa que cada pixel processado requer a combinação de N^2 amostras da imagem de entrada. Para um kernel de 51×51, por exemplo, são necessárias 2.601 operações de combinação para cada pixel sintetizado. Considerando imagens em resolução HD ou Full HD, esse custo torna-se extremamente elevado tanto em processamento quanto em consumo de memória (NIKLAUS; MAI; LIU, 2017; NIKLAUS, 2021).

A técnica de convolução separável busca reduzir essa complexidade explorando uma propriedade matemática específica de determinados filtros. Em vez de representar o kernel como uma matriz bidimensional completa, ele é decomposto em dois filtros unidimensionais independentes: um filtro vertical e um filtro horizontal. Matematicamente, essa decomposição pode ser expressa como:

$$K = v \cdot h^T$$

onde v representa o vetor vertical e h representa o vetor horizontal. Dessa forma, a convolução bidimensional passa a ser executada em duas etapas sucessivas de convolução unidimensional (NIKLAUS; MAI; LIU, 2017).

Como consequência, o número de operações deixa de crescer quadraticamente e passa a crescer linearmente em relação ao tamanho do filtro. Enquanto uma convolução tradicional exige aproximadamente N^2 operações por pixel, a versão separável requer apenas $N + N = 2N$ operações. Para um filtro de dimensão 51×51, isso representa uma redução de 2.601 para apenas 102 operações por pixel.

Essa diferença possui impacto direto na viabilidade computacional do algoritmo. A redução do número de coeficientes diminui a quantidade de acessos à memória, reduz o volume de dados intermediários armazenados durante a inferência e aumenta o potencial de paralelização em arquiteturas modernas de GPU. Como resultado, convoluções separáveis tornam possível utilizar kernels de grande alcance espacial sem que o custo computacional se torne proibitivo (NIKLAUS; MAI; LIU, 2017; KIRK; HWU, 2023).

No contexto deste trabalho, essa propriedade é particularmente relevante, pois a redução da complexidade matemática influencia diretamente a latência de execução e o consumo de memória VRAM durante a inferência. Dessa forma, a separabilidade dos kernels constitui o principal mecanismo que viabiliza a execução de modelos de interpolação de quadros em GPUs de consumo com capacidade limitada de memória.

2.2.5 Implementação de Baixo Nível e I/O de Vídeo

O desenvolvimento de aplicações de processamento de vídeo em alto desempenho exige atenção não apenas aos algoritmos empregados, mas também à forma como os dados são organizados e transferidos entre os diferentes níveis da arquitetura computacional. Nesse contexto, a linguagem C++ é amplamente utilizada em aplicações de visão computacional e computação de alto desempenho por oferecer controle explícito sobre a alocação de memória, estruturas de dados e gerenciamento de recursos do sistema (KIRK; HWU, 2023).

Em aplicações aceleradas por GPU, os dados normalmente são armazenados inicialmente na memória principal do sistema (Host) e posteriormente transferidos para a memória de vídeo da placa gráfica (Device). Essas transferências ocorrem através do barramento PCI Express (PCIe), cuja largura de banda é significativamente inferior à largura de banda interna da memória da GPU. Consequentemente, movimentações excessivas de dados podem se tornar um gargalo relevante para o desempenho da aplicação (NVIDIA Corporation, 2025).

Para minimizar esse problema, é comum organizar os dados em regiões contíguas de memória antes da transferência para o dispositivo gráfico. Essa estratégia melhora a localidade espacial dos acessos, reduz a fragmentação dos dados e favorece operações de leitura e escrita mais eficientes tanto na CPU quanto na GPU (KIRK; HWU, 2023).

No contexto deste trabalho, os quadros de vídeo serão inicialmente carregados utilizando bibliotecas especializadas de entrada e saída de mídia. Após a decodificação, os dados serão reorganizados em estruturas lineares adequadas para processamento paralelo, permitindo transferências eficientes entre memória principal e memória gráfica. Essa etapa é particularmente importante em aplicações de interpolação de quadros, uma vez que múltiplas imagens de alta resolução precisam ser mantidas simultaneamente em memória durante o processo de inferência.

2.2.6 Inferência Desacoplada via ONNX Runtime

O Open Neural Network Exchange (ONNX) consiste em um formato intermediário padronizado para representação de modelos de aprendizado profundo, permitindo interoperabilidade entre diferentes frameworks de treinamento e ambientes de inferência (Linux Foundation AI; Microsoft, 2026). Nesse ecossistema, o ONNX Runtime atua como um mecanismo otimizado de execução, possibilitando a inferência de redes neurais sem dependência direta do framework originalmente utilizado durante o treinamento.

No contexto desta pesquisa, a utilização do ONNX Runtime possui relevância arquitetural, pois permite desacoplar a inferência do ecossistema Python/PyTorch, reduzindo *overheads* relacionados à abstração de alto nível e ao gerenciamento adicional de memória. Além disso, a biblioteca oferece integração nativa com provedores de execução CUDA, permitindo que os tensores permaneçam alocados diretamente na VRAM da GPU durante o pipeline de processamento.

Essa abordagem é particularmente importante para a hipótese investigada neste trabalho, uma vez que a redução de transferências desnecessárias entre memória principal e memória de vídeo constitui um fator relevante para mitigação do gargalo conhecido como *Memory Wall*. Dessa forma, o ONNX Runtime atua não apenas como ferramenta de execução, mas como componente estrutural da estratégia de otimização proposta.

2.2.7 Arquitetura de Decomposição e Garantia de Separabilidade

A viabilidade técnica de utilizar kernels de grandes dimensões (como 51×51) depende da garantia matemática de separabilidade. Para que uma matriz K seja separável, ela deve obrigatoriamente possuir Posto 1 (*Rank-1*), permitindo sua decomposição no produto externo de dois vetores unidimensionais: $K = v \cdot h^T$.

O modelo SepConv resolve esse desafio diretamente em sua arquitetura de rede neural através de uma **bifurcação na camada de saída**. Em vez de estimar os N^2 elementos de uma matriz bidimensional, a rede possui duas ramificações independentes que inferem, simultaneamente, um vetor vertical v $(N \times 1)$ e um vetor horizontal h $(1 \times N)$.

Ao forçar essa estrutura, o algoritmo garante que o kernel resultante seja sempre separável, eliminando erros de aproximação que ocorreriam em uma decomposição analítica

a posteriori. Para a implementação em C++, essa característica é fundamental: o sistema recebe da rede neural apenas $2N$ valores por pixel, permitindo que a operação de convolução seja executada em dois passos lineares sequenciais, reduzindo drasticamente o esforço computacional e a ocupação de memória sem perda de precisão matemática em relação ao modelo bidimensional pretendido.

2.2.8 Arquitetura de GPU e o Gargalo de Memória (*Memory Wall*)

A execução de modelos profundos de visão computacional em hardware de consumo não é apenas um desafio aritmético, mas fundamentalmente um problema de tráfego de dados. Segundo a teoria arquitetural apresentada por Hennessy e Patterson (2022), os sistemas modernos sofrem com o chamado *Memory Wall* (Muro da Memória), um fenômeno no qual a capacidade de processamento cresce em ritmo significativamente superior ao aumento da largura de banda da memória. Como consequência, em diversas aplicações computacionais, especialmente aquelas intensivas em dados, o tempo gasto movimentando informações entre memória e unidades de processamento frequentemente supera o tempo efetivamente dedicado aos cálculos.

No ecossistema de Unidades de Processamento Gráfico (GPUs), essa dinâmica é regida pelo modelo de execução *Single Instruction, Multiple Threads* (SIMT). Conforme descrito por Kirk e Hwu (2023), a eficiência máxima de uma GPU é alcançada quando os acessos à memória ocorrem de forma coalescida e contígua, permitindo que múltiplas threads compartilhem operações de leitura e escrita de maneira eficiente. Transferências excessivas pelo barramento PCI Express (PCIe), bem como realocações frequentes de grandes tensores na memória global da GPU, podem introduzir gargalos significativos e reduzir a taxa efetiva de processamento (NVIDIA Corporation, 2025).

Dessa forma, a escolha metodológica de transpor kernels tradicionais para convoluções separáveis — reduzindo o armazenamento temporal de $O(N^2)$ para $O(N)$ — transcende a mera economia de operações matemáticas de ponto flutuante. A separabilidade matemática reduz o *footprint* (pegada) espacial da inferência diretamente na memória global da GPU, diminuindo a quantidade de dados movimentados e reduzindo a pressão sobre os subsistemas de memória. Essa característica é particularmente relevante considerando que, essa característica é particularmente relevante considerando que, segundo a Steam Hardware Survey, placas de vídeo com 8 GB de VRAM representam atualmente a configuração individual mais comum entre os usuários da plataforma (Valve Corporation, 2026). Dessa forma, a redução do consumo de memória torna-se um requisito prático para ampliar a acessibilidade de técnicas avançadas de interpolação de quadros em ambientes domésticos e em hardware de consumo.

Consequentemente, a redução do consumo de memória proporcionada pelas convoluções separáveis não apenas melhora a eficiência computacional, mas também contribui para evitar limitações impostas pela capacidade física da VRAM, fundamentando teoricamente

a hipótese de redução de latência e viabilização da inferência em GPUs comerciais de capacidade moderada.

2.2.9 Métricas de Avaliação Objetiva de Qualidade

A validação de algoritmos de Interpolação de Quadros de Vídeo (VFI) exige a comparação entre o quadro sintetizado (I_{sint}) e o quadro real de referência (I_{real}), também denominado *ground truth*. Para quantificar objetivamente a fidelidade visual das imagens geradas, este trabalho utilizará duas métricas amplamente adotadas na literatura de processamento de imagens e visão computacional: Peak Signal-to-Noise Ratio (PSNR) e Structural Similarity Index (SSIM).

1. Peak Signal-to-Noise Ratio (PSNR):

O PSNR é uma das métricas mais tradicionais para avaliação da qualidade de imagens reconstruídas ou comprimidas. Sua formulação é baseada no Erro Quadrático Médio (*Mean Squared Error* – MSE), que mede a diferença pixel a pixel entre a imagem sintetizada e a imagem de referência (HORE; ZIOU, 2010).

Matematicamente, o PSNR é definido como:

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \tag{2.1}$$

onde MAX_I representa o valor máximo possível de intensidade de um pixel, assumindo normalmente o valor 255 em imagens de 8 bits.

Valores mais elevados de PSNR indicam menor erro de reconstrução e, consequentemente, maior proximidade numérica entre a imagem gerada e a imagem original. Entretanto, embora seja simples e computacionalmente eficiente, essa métrica apresenta limitações perceptuais, pois considera apenas diferenças absolutas entre pixels, sem levar em conta a forma como o sistema visual humano interpreta estruturas, texturas e padrões espaciais (HORE; ZIOU, 2010).

2. Structural Similarity Index (SSIM):

Para contornar as limitações perceptuais do PSNR, Wang et al. (2004) propuseram o *Structural Similarity Index* (SSIM), uma métrica baseada na hipótese de que o sistema visual humano é mais sensível à preservação de estruturas locais do que a diferenças absolutas de intensidade.

O SSIM compara regiões correspondentes entre duas imagens considerando simultaneamente três componentes fundamentais:

- luminância;
- contraste;
- estrutura.

O resultado é um valor normalmente compreendido no intervalo $[0, 1]$, onde valores próximos de 1 indicam elevada similaridade estrutural entre as imagens comparadas.

Em aplicações de interpolação de quadros, o SSIM é particularmente relevante por ser capaz de identificar degradações visuais frequentemente observadas em quadros sintetizados, como borramento de bordas, perda de detalhes finos e artefatos decorrentes de erros de estimativa de movimento. Por esse motivo, a métrica é amplamente utilizada na literatura contemporânea de VFI como complemento ao PSNR (NIKLAUS; MAI; LIU, 2017; NIKLAUS, 2021).

A utilização conjunta de PSNR e SSIM permite avaliar tanto a precisão numérica quanto a qualidade perceptual das imagens geradas. Dessa forma, torna-se possível analisar os impactos das otimizações propostas neste trabalho não apenas sobre a latência e o consumo de memória, mas também sobre a fidelidade visual dos quadros sintetizados.

2.2.10 Mensuração de Desempenho Computacional

Além da qualidade visual dos quadros sintetizados, a avaliação de algoritmos de Interpolação de Quadros de Vídeo (VFI) envolve aspectos relacionados à eficiência computacional. Em aplicações práticas, especialmente em sistemas de tempo real, a qualidade da imagem deve ser analisada em conjunto com o custo necessário para produzi-la, uma vez que algoritmos visualmente precisos podem apresentar inviabilidade operacional devido ao elevado consumo de recursos computacionais (KIRK; HWU, 2023).

Nesse contexto, duas métricas são frequentemente empregadas na literatura para caracterizar o desempenho de sistemas acelerados por GPU: a latência de processamento e o consumo de memória. A latência representa o tempo necessário para que uma determinada operação seja concluída, influenciando diretamente a capacidade do sistema de processar sequências de vídeo em tempo real. Em aplicações de visão computacional, reduções de poucos milissegundos por quadro podem resultar em aumentos significativos na taxa efetiva de processamento (*throughput*) (HENNESSY; PATTERSON, 2022).

Por sua vez, o consumo de memória constitui um fator limitante para a execução de modelos de aprendizado profundo em hardware de consumo. A necessidade de armazenar simultaneamente quadros de entrada, ativações intermediárias e parâmetros do modelo pode levar ao esgotamento da memória disponível, inviabilizando a inferência mesmo quando a capacidade de processamento da GPU permanece ociosa. Por esse motivo, a utilização eficiente da VRAM tornou-se um aspecto relevante no desenvolvimento de aplicações modernas de inteligência artificial (NVIDIA Corporation, 2025).

No contexto da interpolação de quadros baseada em convoluções adaptativas separáveis, a redução da complexidade espacial representa um dos principais fatores que podem contribuir para a diminuição do consumo de memória e para a melhoria da eficiência geral do sistema. Dessa forma, a análise conjunta de métricas de qualidade visual e desempenho

computacional permite uma avaliação mais abrangente dos benefícios e limitações da abordagem investigada neste trabalho.

2.3 Trabalhos relacionados

Nesta seção, são apresentados três trabalhos recentes que abordam o desafio da Interpolação de Quadros de Vídeo (VFI). A seleção prioriza métodos que, assim como esta pesquisa, buscam o equilíbrio entre a fidelidade visual e a eficiência no processamento de grandes deslocamentos de pixels.

2.3.1 Trabalho 1: Revisiting Adaptive Convolutions for Video Frame Interpolation

Niklaus (2021) abordou o problema da complexidade crescente das redes neurais de interpolação, que muitas vezes exigem recursos computacionais excessivos para ganhos marginais de qualidade. Para resolvê-lo, o autor propôs uma revisitação ao método de convoluções adaptativas, introduzindo melhorias sutis na arquitetura de estimativa de kernel. A metodologia baseou-se no refinamento do modelo SepConv original, otimizando a forma como os filtros $1D$ são gerados. Os resultados mostraram que modelos baseados em kernels podem atingir métricas de PSNR e SSIM próximas ao estado da arte com estruturas mais simples. No entanto, a solução ainda depende de frameworks de alto nível (PyTorch) para execução. O presente trabalho diferencia-se por investigar experimentalmente a inferência pura baseada nesses conceitos em C++, focando na eliminação do *overhead* inerente às bibliotecas de aprendizado profundo na alocação de GPU.

2.3.2 Trabalho 2: Motion-Aware Video Frame Interpolation

Han et al. (2024) abordaram o problema da representação inadequada de movimentos complexos em métodos tradicionais de interpolação de quadros, especialmente em cenários com deslocamentos rápidos, mudanças abruptas de direção e regiões de oclusão. Para enfrentar essa limitação, os autores propuseram uma arquitetura baseada em modelagem de movimento multi-escala, capaz de capturar informações espaciais e temporais em diferentes níveis de resolução.

A metodologia combina a extração hierárquica de características com mecanismos de estimativa de movimento que integram informações provenientes de múltiplas escalas da imagem. Essa estratégia permite representar movimentos de grande amplitude sem depender exclusivamente de kernels locais ou de campos de fluxo óptico densos, reduzindo artefatos temporais frequentemente observados em sequências complexas.

Os resultados experimentais demonstraram melhorias na preservação da consistência temporal e na qualidade visual dos quadros sintetizados quando comparados a métodos

anteriores de interpolação. Em particular, os autores reportam redução de artefatos em regiões com movimentos rápidos e melhor reconstrução de detalhes estruturais.

Entretanto, a arquitetura proposta prioriza a qualidade visual e a robustez da representação de movimento, sem foco específico na redução do consumo de memória ou na otimização da inferência para hardware de consumo. O presente trabalho diferencia-se ao investigar experimentalmente como a separabilidade de kernels e a inferência desacoplada via ONNX Runtime podem contribuir para reduzir a latência e o consumo de VRAM em GPUs comerciais, mantendo níveis aceitáveis de fidelidade visual.

2.3.3 Trabalho 3: RAFT: Recurrent All-Pairs Field Transforms for Optical Flow

Teed e Deng (2020) abordou a precisão na estimativa de movimento, um problema seminal para a síntese de novos quadros. Para resolvê-lo, propuseram o modelo RAFT, que utiliza uma arquitetura recorrente para atualizar campos de fluxo óptico através de volumes de correlação de todos os pares de pixels. A metodologia utiliza iterações sucessivas para refinar o movimento detectado. Os resultados estabeleceram um novo estado da arte em termos de precisão de vetores. No entanto, a solução apresenta alta latência e exige GPUs de alto desempenho, limitando severamente a viabilidade em cenários comuns. O presente trabalho diferencia-se por adotar a convolução adaptativa como uma alternativa mais contida ao fluxo óptico denso do RAFT, priorizando a performance em GPU e a otimização do consumo de VRAM através de uma inferência nativa.

Tabela 1 – Comparação entre trabalhos relacionados e a abordagem proposta

Trabalho	**Estratégia VFI**	**Tecnologia**	**Foco / Diferencial**
Niklaus (2021)	Convolução Adaptativa Refinada	Redes Neurais (PyTorch)	Qualidade visual elevada com arquitetura baseada em kernels adaptativos separáveis.
Han et al. (2024)	Modelagem de Movimento Multi-Escala	Pirâmide Hierárquica e Fluxo Intermediário	Melhor representação de movimentos complexos e redução de artefatos temporais.
RAFT (2020)	Fluxo Óptico Recorrente	Volumes de Correlação	Alta precisão na estimativa de movimento com elevado custo computacional.
Este trabalho	**Inferência SepConv em Baixo Nível**	**C++ / CUDA / ONNX Runtime**	**Redução de latência e consumo de VRAM em GPUs de consumo mantendo qualidade visual adequada.**

Fonte: Elaborado pelo autor (2026)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para o desenvolvimento completo do projeto no TCC2.

3.1 Solução proposta

A solução proposta para investigar a viabilidade técnica e a eficiência das convoluções adaptativas separáveis consiste no desenvolvimento de um motor de inferência customizado para Interpolação de Quadros de Vídeo (VFI). O foco é transpor a etapa de inferência do modelo SepConv para um ambiente otimizado em GPU. A descrição detalhada do projeto segue abaixo:

- **Visão geral da solução**: Será desenvolvida uma ferramenta de software de linha de comando (CLI) focada puramente na inferência do modelo SepConv. O sistema processará pares de quadros de um vídeo de entrada, utilizará pesos pré-treinados para gerar os *kernels* de movimento, sintetizará o quadro intermediário e compilará um novo vídeo de saída, registrando os picos de uso de VRAM integrando a API da *NVIDIA Management Library* (NVML) e a latência da execução via eventos CUDA.
- **Arquitetura**: O sistema opera em um fluxo de dados (*pipeline*) de cinco estágios:
  1. **I/O (Host):** Leitura e decodificação dos quadros na RAM do sistema via OpenCV;
  2. **Transferência (Host-to-Device):** Envio dos dados linearizados da RAM para a VRAM da GPU;
  3. **Inferência (ONNX Runtime):** Processamento do modelo pré-treinado SepConv para gerar os vetores separados 1D para cada pixel, mantendo-os alocados nativamente na GPU;
  4. **Convolução (CUDA):** Aplicação paralela da convolução separável $O(N)$ utilizando blocos e *threads* CUDA nativamente na VRAM para compor o quadro final;
  5. **Retorno e Escrita:** Transferência apenas do quadro gerado final de volta para o Host e codificação do vídeo.
- **Tecnologias e ferramentas**: O controle do fluxo será desenvolvido em **C++17**. Para a leitura e decodificação do vídeo, optou-se pela biblioteca **OpenCV (C++**

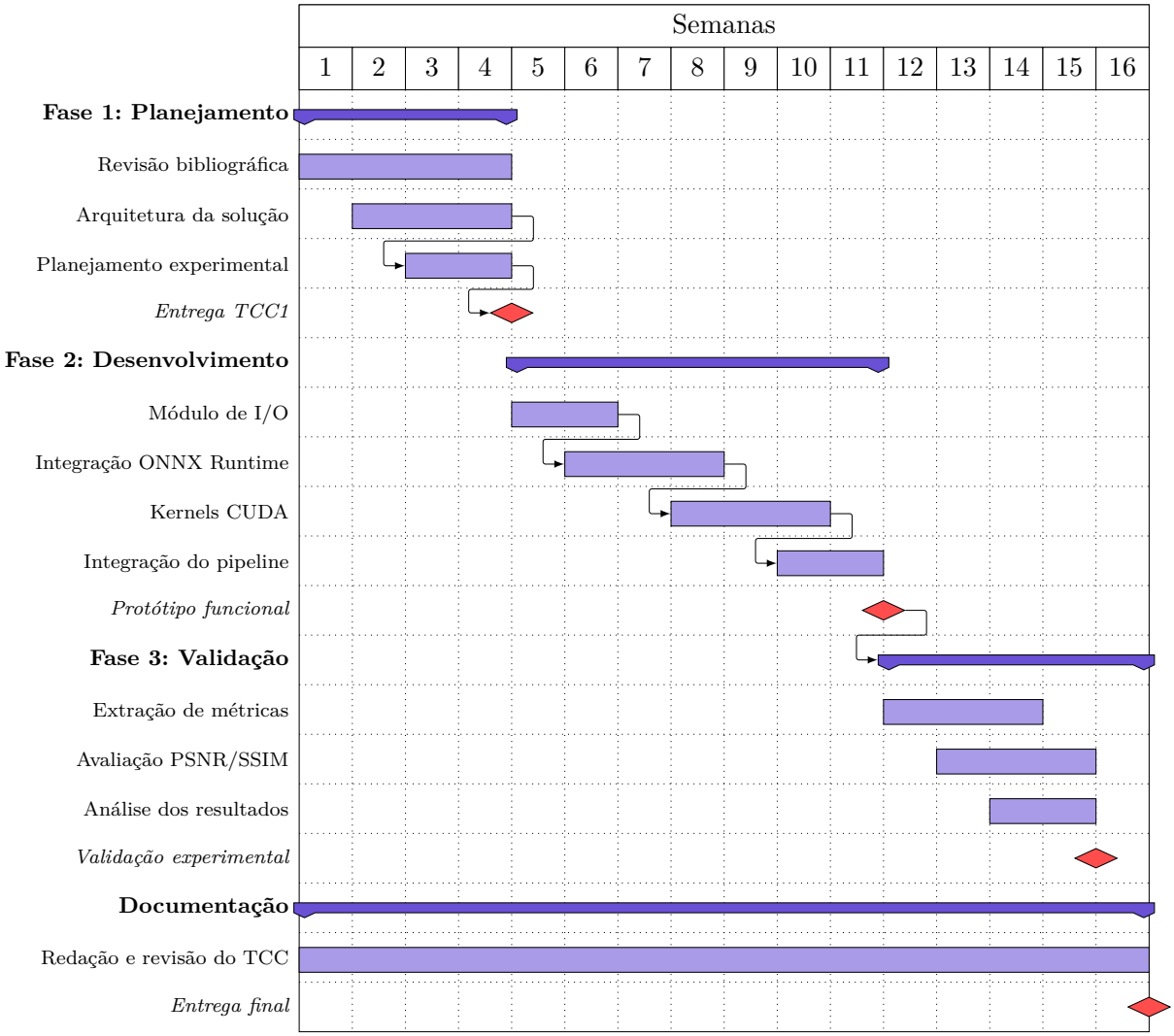
API). O motor de inferência dos filtros será a biblioteca **ONNX Runtime** (operando via provedor CUDA), enquanto a aplicação matemática intensiva e paralela sobre a imagem fará uso das APIs nativas **NVIDIA CUDA C++**.

- **Funcionalidades principais**: A ferramenta carregará o modelo '.onnx', executará a predição quadro a quadro, aferirá os tempos de latência em milissegundos utilizando as medições do ecossistema NVIDIA e exportará um consolidado do pico de memória VRAM registrado.
- **Metodologia de desenvolvimento**: Será adotada uma abordagem de ciclos iterativos de otimização (*Profiling-Driven Development*). O protótipo validará as transferências básicas de memória e, progressivamente, focará em medir o comportamento da placa de vídeo e o esgotamento dos seus 8 GB ao aplicar o kernel de forma escalonada.
- **Critérios de validação e Mensuração de Hardware**: O sucesso da eficiência será avaliado contrastando os resultados da arquitetura proposta com a *baseline* padrão em Python/PyTorch do mesmo modelo. A coleta de dados do consumo de VRAM será efetuada através da *NVIDIA Management Library* (NVML) e utilitários como o *Nsight Systems*, garantindo a precisão ao isolar o processamento principal. A latência será aferida no código C++ através de eventos da API (`cudaEventRecord`), isolando o tempo real de processamento matemático paralelo na GPU.
- **Seleção de Dataset**: A avaliação experimental de qualidade visual e consumo associado será conduzida usando fragmentos do *dataset* **Vimeo-90K**, escalonados e ajustados para a resolução de testes.

3.2 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Figura 2 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

Referências

BAO, W. et al. Depth-aware video frame interpolation. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2019. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/1904.00830>. 5

BORDWELL, D.; THOMPSON, K.; SMITH, J. *Film Art: An Introduction*. 11. ed. [S.l.]: McGraw-Hill Education, 2016. 4

GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3. ed. São Paulo: Pearson, 2009. 5, 10, 11

HAN, P. et al. Motion-aware video frame interpolation. *Neural Networks*, v. 178, p. 106433, 2024. ISSN 0893-6080. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0893608024003575>. 19

HENNESSY, J. L.; PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 7. ed. [S.l.]: Morgan Kaufmann, 2022. 16, 18

HORE, A. C.; ZIOU, D. Image quality metrics: Psnr vs. ssim. *20th International Conference on Pattern Recognition*, p. 2366–2369, 2010. 17

KIRK, D. B.; HWU, W. mei W. *Programming Massively Parallel Processors: A Hands-on Approach*. 5. ed. [S.l.]: Morgan Kaufmann, 2023. 11, 14, 16, 18

Linux Foundation AI; Microsoft. *ONNX Runtime Documentation*. 2026. Acesso em: 28 maio 2026. Disponível em: <https://onnxruntime.ai/>. 15

NIKLAUS, S. Revisiting adaptive convolutions for video frame interpolation. In: *IEEE Winter Conference on Applications of Computer Vision (WACV)*. [s.n.], 2021. Acesso em: 12 abr. 2026. Disponível em: <https://openaccess.thecvf.com/content/WACV2021/html/Niklaus_Revisiting_Adaptive_Convolutions_for_Video_Frame_Interpolation_WACV_2021_paper.html>. 12, 13, 18, 19

NIKLAUS, S.; MAI, L.; LIU, F. Video frame interpolation via adaptive separable convolution. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2017. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/1708.01692>. 5, 6, 12, 13, 14, 18

NVIDIA Corporation. *CUDA C++ Programming Guide*. 2025. Acesso em: 15 maio 2026. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. 14, 16, 18

SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2. ed. [S.l.]: Springer, 2022. 11, 12

TEED, Z.; DENG, J. Raft: Recurrent all-pairs field transforms for optical flow. In: *European Conference on Computer Vision (ECCV)*. [s.n.], 2020. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/2003.12039>. 4, 12, 20

Valve Corporation. *Steam Hardware & Software Survey*. 2026. Acesso em: 28 maio 2026. Disponível em: <https://store.steampowered.com/hwsurvey>. 6, 16

WANG, Z. et al. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, IEEE, v. 13, n. 4, p. 600–612, 2004. 17

WATKINSON, J. *The Art of Digital Video*. 4. ed. [S.l.]: Focal Press, 2009. 4